OS Assignments

Assignment 3

1. Display Calendar of year 2026.

```
cal 2026
```

2. Display the system's date.

```
date
```

3. Count the number of lines in the /etc/passwd file.

```
wc -l /etc/passwd
```

4. Find out who else is on the system.

```
who
```

5. Display Calendar of current month.

```
cal
```

6. Create a subdirectory called mydir.

```
mkdir mydir
```

7. Move the file mydate into the new subdirectory.

```
mv mydate mydir/
```

8. Go to the subdirectory mydir and copy the file mydate to a new file called ourdate.

```
cd mydir
cp mydate ourdate
```

9. List the contents of mydir.

```
ls mydir
```

10. Do a long listing on the file ourdate and note the permissions.

```
ls -l ourdate
```

11. Display the name of the current directory starting from the root.

```
pwd
```

12. Move the files in the directory mydir back to your home directory.

```
mv * ~
```

13. Display the first 5 lines of mydate.

```
head -5 mydate
```

14. Display the last 8 lines of mydate.

```
tail -8 mydate
```

15. Remove the directory mydir.

```
rmdir mydir
```

16. Redirect the output of the long listing of files to a file named list.

```
ls -l > list
```

17. Select any 5 capitals of states in India and enter them in capitals1, 5 more in capitals2, and 5 more in capitals3. Concatenate all 3 files and redirect the output to capitals.

```
cat capitals1 capitals2 capitals3 > capitals
```

18. Concatenate the file capitals2 at the end of file capitals.

```
cat capitals2 >> capitals
```

19. Give read and write permissions to all users for the file capitals.

```
chmod 666 capitals
```

20. Give read permissions only to the owner of the file capitals. Open the file, make some changes and try to save it.

```
chmod 400 capitals
```

Answer: File opens but cannot be saved because write permission is removed.

21. Create an alias to concatenate the 3 files capitals1, capitals2, capitals3 and redirect the output to a file named capitals. Activate the alias and make it run.

```
alias combine='cat capitals1 capitals2 capitals3 > capitals'
combine
```

22. Find out the number of times the string "the" appears in the file mydate.

```
grep -o "the" mydate | wc -l
```

23. Find out the line numbers on which the string "date" exists in mydate.

```
grep -n "date" mydate
```

24. Print all lines of mydate except those that have the letter "i" in them.

```
grep -v "i" mydate
```

25. List the words of 4 letters from the file mydate.

```
grep -o '\b....\b' mydate
```

26. List 5 states in north east India in a file mystates. List their corresponding capitals in a file mycapitals. Use the paste command to join the 2 files.

```
paste mystates mycapitals
```

27. Use the cut command to print the 1st and 3rd columns of the /etc/passwd file.

```
cut -d: -f1,3 /etc/passwd
```

28. Count the number of people logged in and also trap the users in a file using the tee command.

```
who | tee usersfile | wc -l
```

29. Convert the contents of mystates into uppercase.

```
tr 'a-z' 'A-Z' < mystates
```

30. Create any two files and display the common values between them.

```
comm -12 file1 file2
```

Assignment 4

1. Write shell script to execute commands ls, date, pwd repetitively.

```
#!/bin/bash

while true
do
    ls
    date
    pwd
    sleep 2
done
```

2. Write a shell script to assign value to the variable. Display value with and without $.

```
#!/bin/bash

name="Linux"
echo $name
echo \$name
```

3. Variables are untyped in Shell Script. Write a shell script to show variables are untyped.

```
#!/bin/bash

var=10
echo "Integer value: $var"
var="Hello"
echo "String value: $var"
```

4. Write a shell script to accept numbers from user (keyboard).

```
#!/bin/bash

echo "Enter a number:"
read num
echo "You entered: $num"
```

5. Write a shell script to accept numbers from command line arguments.

```
#!/bin/bash

echo "Entered numbers are: $@"
```

6. Write a shell script to show the contents of environmental variables SHELL, PATH, HOME.

```
#!/bin/bash

echo "SHELL=$SHELL"
echo "PATH=$PATH"
echo "HOME=$HOME"
```

7. Write a shell script to create two files. Accept file names from user.

```
#!/bin/bash

echo "Enter first file name:"
read f1
echo "Enter second file name:"
read f2
touch "$f1" "$f2"
echo "Files created successfully"
```

8. Write a shell script to create two directories. Accept directory names from command line.

```
#!/bin/bash

mkdir "$1" "$2"
echo "Directories created successfully"
```

9. Write a shell script to copy file content of one file to another file. Accept file names from command line argument.

```
#!/bin/bash

cp "$1" "$2"
echo "File copied successfully"
```

10. Write a shell script to rename the file name. Accept old filename and new filename from command line argument.

```
#!/bin/bash

mv "$1" "$2"
echo "File renamed successfully"
```

11. Write a shell script to display capitals with serial numbers.

```
#!/bin/bash

nl capitals
```

12. Write a shell script to perform arithmetic operation of float data.

```
#!/bin/bash

echo "Enter two float numbers:"
read a b
echo "Addition = $(echo "$a + $b" | bc)"
echo "Subtraction = $(echo "$a - $b" | bc)"
echo "Multiplication = $(echo "$a * $b" | bc)"
echo "Division = $(echo "scale=2; $a / $b" | bc)"
```

Assignment 5

1. Write a shell script to check number entered by the user is greater than 10.

```
#!/bin/bash

echo "Enter a number:"
read num

if [ "$num" -gt 10 ]
then
    echo "Number is greater than 10"
else
    echo "Number is less than or equal to 10"
fi
```

2. Write a shell script to check if a file exists. If not, then create it.

```
#!/bin/bash

echo "Enter file name:"
read filename

if [ -f "$filename" ]
then
    echo "File already exists."
else
    touch "$filename"
    echo "File created successfully."
fi
```

3. Write a shell script that takes two command line arguments. Check whether the name passed as first argument is of a directory or not. If not, create directory using name passed as second argument.

```
#!/bin/bash

if [ -d "$1" ]
then
    echo "Directory '$1' already exists."
else
    mkdir "$2"
    echo "Directory '$2' created successfully."
fi
```

4. Write a shell script which checks the total arguments passed. If the argument count is greater than 5, then display message "Too many arguments".

```
#!/bin/bash

if [ $# -gt 5 ]
then
    echo "Too many arguments"
else
    echo "Argument count is acceptable"
fi
```

5. Write a shell script to check arguments passed at command line is whether of a file or directory.

```
#!/bin/bash

if [ -f "$1" ]
then
    echo "It is a file."
elif [ -d "$1" ]
then
    echo "It is a directory."
else
    echo "It is neither a file nor a directory."
fi
```

6. Write a shell script to read a month name from the user. Check if the name entered is either Feb or March.

```
#!/bin/bash

echo "Enter month name:"
read month
```

```
if [ "$month" = "Feb" ] || [ "$month" = "March" ]
then
    echo "Month is either Feb or March."
else
    echo "Month is not Feb or March."
fi
```

7. Write a shell script to check whether file or directory exists.

```
#!/bin/bash

echo "Enter file or directory name:"
read name

if [ -f "$name" ]
then
    echo "It is a file and it exists."
elif [ -d "$name" ]
then
    echo "It is a directory and it exists."
else
    echo "File or directory does not exist."
fi
```

8. Write a shell script to check whether file exists and file is readable.

```
#!/bin/bash

echo "Enter file name:"
read filename

if [ -f "$filename" ] && [ -r "$filename" ]
then
    echo "File exists and is readable."
elif [ -f "$filename" ]
then
    echo "File exists but is not readable."
else
    echo "File does not exist"
fi
```

9. Write a shell script to check if the present month is Feb or not. Use date command to get present month.

```
#!/bin/bash

month=$(date +%b)

if [ "$month" = "Feb" ]
```

```
then
    echo "Present month is February."
else
    echo "Present month is not February."
fi
```

10. Write a shell script to check if the current user is root or regular user.

```
#!/bin/bash

if [ "$USER" = "root" ]
then
    echo "Current user is Root user."
else
    echo "Current user is Regular user."
fi
```

11. Write a shell script to check the total arguments passed at command line. If the arguments are more than 3 then list the arguments else print "type more next time".

```
#!/bin/bash

if [ $# -gt 3 ]
then
    echo "Arguments are:"
    echo "$@"
else
    echo "type more next time"
fi
```

Assignment 6

1. Write a shell script to create three files.

```
#!/bin/bash

touch file1 file2 file3
echo "Three files created successfully"
```

2. Write a shell script to print date and time 10 times, once after each second.

```
#!/bin/bash

for i in {1..10}
do
    date
```

```
    sleep 1
done
```

3. Write a shell script to create five directories. Accept the names of directories from command line.

```
#!/bin/bash

mkdir "$1" "$2" "$3" "$4" "$5"
echo "Five directories created successfully"
```

4. Write a shell script to print long list of all file names passed at command line.

```
#!/bin/bash

ls -l "$@"
```

5. Write a shell script to count the number of file names passed at command line.

```
#!/bin/bash

echo "Total file names passed: $#"
```

6. Write a shell script to accept and display array. Assume array consists of 5 numbers.

```
#!/bin/bash

echo "Enter 5 numbers:"
for i in {0..4}
do
    read arr[$i]
done

echo "Array elements are: ${arr[@]}"
```

7. Write a shell script to find out even and odd numbers from accepted array. Assume array consists of 5 numbers. Also accept array from users.

```
#!/bin/bash

echo "Enter 5 numbers:"
for i in {0..4}
do
    read arr[$i]
done
```

```
for num in "${arr[@]}"
do
    if [ $((num % 2)) -eq 0 ]
    then
        echo "$num is Even"
    else
        echo "$num is Odd"
    fi
done
```

8. Write a shell script to find out the reverse number of a given number.

```
#!/bin/bash

echo "Enter a number:"
read num
rev=0

while [ $num -gt 0 ]
do
    rem=$((num % 10))
    rev=$((rev * 10 + rem))
    num=$((num / 10))
done

echo "Reverse number is: $rev"
```

9. Write a shell script to sort array numbers in ascending and descending order. Assume array consists of 5 numbers. Also accept array from users.

```
#!/bin/bash

echo "Enter 5 numbers:"
for i in {0..4}
do
    read arr[$i]
done

echo "Ascending order:"
printf '%s\n' "${arr[@]}" | sort -n

echo "Descending order:"
printf '%s\n' "${arr[@]}" | sort -nr
```

10. Write a shell script to find out smallest number and largest number of given array. Assume array consists of 5 numbers. Also accept array from users.

```
#!/bin/bash

echo "Enter 5 numbers:"
for i in {0..4}
do
    read arr[$i]
done

smallest=${arr[^0]}
largest=${arr[^0]
<span style="display:none">[^1][^2][^3][^4][^5]</span>

<div align="center">⁂</div>

[^1]: OS-ASSIGNMENT-_5_Om_Kharate_CS_F_B3_62.pdf
[^2]: OS-ASSIGNMENT-_4_Om_Kharate_CS_F_B3_62.docx
[^3]: OS-ASSIGNMENT-_6_Om_Kharate_CS_F_B3_62.docx
[^4]: OS-ASSIGNMENT-3_Om_Kharate_CS_F_B3_62.docx
[^5]: OS-ASSIGNMENT-_5_Om_Kharate_CS_F_B3_62.docx
```